

СОДЕРЖАНИЕ

Введение	2
1. Теоретические основы обезличивания данных.....	3
1.1 Понятие и классификация персональных данных.....	3
1.2 Понятие и классификация обезличивания данных	4
1.3 Цели и задачи обезличивания данных	5
1.4 Методы обезличивания данных	5
1.5 Применение методов обезличивания в различных сферах.....	8
1.6 Проблемы в области обезличивания данных.....	8
1.7 Выводы по главе	10
2. Анализ существующих систем.....	11
2.1 Системы для обезличивания данных	11
2.2 Выводы по главе	14
3. Практическая работа и технологические задачи	15
3.1 Анализ требований	15
3.2 Проектирование программного обеспечения	16
3.2.1 Описание классов	16
3.2.2 Архитектура программного обеспечения.....	20
3.2.3 Алгоритм решения задачи обезличивания данных	23
3.3 Интерфейс программной системы.....	24
3.4 Выводы по главе.....	31
Заключение.....	32
Список использованных источников.....	33

ВВЕДЕНИЕ

С увеличением объемов обрабатываемых данных возникает серьезная угроза утечек личной информации, что ставит под угрозу приватность пользователей. Обезличивание данных является важным инструментом защиты конфиденциальности, позволяя использовать данные для анализа и обработки без раскрытия личных данных субъектов. Существующие методы обезличивания данных требуют постоянного совершенствования и адаптации к новым вызовам в области информационной безопасности.

Нарушения конфиденциальности и утечка личной информации становятся важной проблемой, как для отдельных пользователей, так и для организаций. Законы, такие как GDPR в Европе или российский Закон о персональных данных, требуют от организаций соблюдения строгих норм безопасности при обработке данных. В этом контексте обезличивание данных играет ключевую роль в снижении рисков утечек и повышении доверия со стороны пользователей. Актуальность темы особенно высока в свете роста использования методов машинного обучения и искусственного интеллекта, где обезличенные данные необходимы для обучения алгоритмов и принятия решений.

Целью данной практики является написание реферата по теме «Системы и алгоритмы обезличивания данных». Для этого необходимо изучить существующие подходы к обезличиванию данных, особое внимание следует уделить вопросам конфиденциальности и безопасности данных, а также международным стандартам и регуляторным требованиям в этой области.

В рамках практики необходимо также составить описание кафедры вычислительной техники НГТУ, включая современное состояние материально-технической базы, организацию учебного процесса и научно-исследовательской работы на кафедре.

Результатом практики станет формирование библиографического списка, пригодного для использования в магистерской диссертации, а также описание ключевых подходов к существующим системам и алгоритмам обезличивания данных с их преимуществами и недостатками.

1. Теоретические основы обезличивания данных

Обезличивание данных — это процесс изменения данных таким образом, чтобы они больше не могли быть использованы для идентификации конкретного субъекта. Основная цель обезличивания заключается в защите конфиденциальности и снижении рисков, связанных с утечками личной информации, при этом сохраняя возможность анализа данных. В этой главе рассматриваются основные методы обезличивания, их принципы и применение в различных сферах [2].

1.1 Понятие и классификация персональных данных

Персональные данные – любая информация, относящаяся к прямо или косвенно определенному физическому лицу и хранимая в информационных системах в электронном виде.

Персональные данные могут быть различных типов и зависят от конкретной ситуации, в которой они используются.

Ниже приведены некоторые из наиболее распространенных типов данных:

1. Идентификационные данные - информация, которая позволяет идентифицировать человека, например, его полное имя, адрес, номер телефона, номер и серия паспорта и пр.
2. Финансовые данные - информация, связанная с финансовыми операциями, например, банковские реквизиты, номера кредитных карт и другие данные, которые используются для совершения платежей.
3. Медицинские данные - информация о здоровье человека, т.е. результаты анализов, история болезней, рецепты, лекарства, медицинские диагнозы и пр.
4. Генетические данные - информация о генетических особенностях человека, которая может быть получена из его ДНК, например, группа крови, наследственные заболевания и пр.
5. Данные о «поведении в глобальной сети «Интернет» - информация о действиях и предпочтениях пользователя, например, посещенных сайтах, покупках, просмотренных видео и пр.

6. Биометрические данные - информация, которая может быть получена из биометрических характеристик человека, например, снимков лица или отпечатков пальцев

Данный список не является полным, однако он демонстрирует широкий спектр информации, которая может быть признана персональными данными. Конкретные типы данных могут различаться в зависимости от того, как они используются, а также - как они могут быть связаны с конкретным индивидом [3].

1.2 Понятие и классификация обезличивания данных

Обезличивание данных является важной частью стратегии защиты персональной информации. Основной задачей этого процесса является удаление или модификация таких данных, которые могут быть использованы для идентификации конкретного человека. В результате обезличивания данные становятся безопасными для обработки и использования в аналитических и исследовательских целях, при этом они теряют свою идентифицируемость [3].

Методы обезличивания можно разделить на два основных типа:

- Методы анонимизации
- Методы псевдоанонимизации

Данные считаются анонимизированными в случае, когда невозможно установить обратную персонификацию (необратимое обезличивание). В то время как псевдоанонимизация предполагает создание правил или таблиц, на основе которых можно восстановить изначальные данные

Данные типы обезличивания данных обладают характерными алгоритмами, за счет которого обеспечивается секретность. Для псевдоанонимизации это создание таблиц связей, для анонимизации это алгоритмы удаления данных. К общим можно отнести алгоритмы модификации данных и исходных таблиц, которые могут как обладать обратимостью, так и нет [4].

1.3 Цели и задачи обезличивания данных

Ключевой целью обезличивания персональных данных является защита конфиденциальной информации о субъектах и обеспечение безопасности при выполнении различных задач, связанных с обработкой таких данных. Эти задачи включают:

- обработку и исследования персональных данных;
- сбор и хранение персональных данных;
- обработку поисковых запросов;
- актуализацию персональных данных;
- интеграцию от различных операторов;
- ведение учёта субъектов персональных данных.

Само обезличивание персональных данных преследует несколько важных пунктов:

- выделение и описание признаков субъектов в информационных базах данных, содержащих конфиденциальную информацию;
- выбор методов и программно-инструментальных средств для обезличивания;
- предварительная обработка информационных баз данных;
- обезличивание информационных баз данных с использованием выбранных методов и средств, формирование обезличенных баз данных;
- оценка эффективности обезличивания данных (оценка рисков повторной идентификации обезличенных баз данных и оценка информационных потерь);
- документирование основных этапов обезличивания и полученных результатов, описание обезличенных баз данных [5].

1.4 Методы обезличивания данных

Для эффективного обезличивания данных используются различные методы, каждый из которых подходит для определенных задач и типов данных.

Среди них можно выделить четыре самых распространённых метода:

- Введение идентификаторов

Вместо использования реальных идентификаторов (например, имен или номеров социального страхования) используются уникальные коды, которые не позволяют установить привязку к конкретному субъекту.

- Перемешивание (перестановка)

Порядок записей в данных меняется случайным образом, чтобы нарушить связь между индивидуальными атрибутами. Секретность обеспечивается алгоритмом перемещения данных в исходной таблице.

- Декомпозиция

Данные разбиваются на меньшие фрагменты, чтобы исключить их объединение в единый набор, позволяющий идентифицировать человека. Информация о связях хранится в отдельной таблице. Секретность обеспечивается за счет создания таблицы связей.

- Изменение состава или семантики

Значения данных изменяются таким образом, чтобы сохранить общую структуру набора данных, но нарушить идентификацию. Секретность обеспечивается алгоритмом модификации данных [6].

Существуют также и другие, менее используемые методы. Среди них:

- Глобальное и локальное обобщение
- Кодирование сверху и/или снизу
- Локальное подавление
- Микроагрегирование
- Добавление шума
- Округление
- Маскирование по шаблону
- Случайная выборка
- Удаление

Также на данный момент существуют только совсем редкие методы

обезличивания, которые как правило находятся еще на стадии ранней разработки и не нашли широкого применения в данной сфере. Это как правило методы, основанные на искусственных нейронных сетях и синтетических алгоритмах. Однако они являются довольно перспективными и должны быть рассмотрены в рамках будущей дипломной работы. Так как псевдоанонимизация персональных данных является задачей схожей с задачами криптографии, предлагается рассмотреть алгоритм, основанный на синхронизации двух искусственных нейронных сетей. Данный алгоритм предполагает создание нейросетей, основанных на одинаковых обучающих наборах, что позволяет одной из них засекречивать данные, а другой – расшифровывать их. При чем, данный подход показал высокую криптостойкость, позволяющую обеспечить полную безопасность хранимых и передаваемых данных [6].

Синтетические методы обезличивания предполагают замену существующих реальных значений в базах данных значениями синтетическими, по структуре сходными с первоначальными.

Методы синтетического обезличивания данных включают в себя построение математических моделей на основе шаблонов, содержащихся в исходном наборе данных. Методы синтетического обезличивания опираются на типовой статистический инструмент: частотные оценки вероятностей, стандартные отклонения, линейную регрессию, медианы, статистические средние величины, размахов выборок или другие статистические методы.

Метод синтеза хорошо показывают себя в задаче обезличивания данных, особенности которых требуют для обезличивания больших усилий. При этом поддерживается высокий уровень анонимности. Даже самые простые статистические методы предлагают довольно универсальные решения для большинства возможных числовых атрибутов. Но несмотря на очевидную привлекательность метода синтеза, в исследованиях ему уделяется довольно мало внимания [7].

Рассмотренные методы могут использоваться как по отдельности, так и

комбинированно, что позволяет сочетать необходимый уровень безопасности, скорости обработки и обратимости исходных данных.

1.5 Применение методов обезличивания в различных сферах

Методы обезличивания данных находят широкое применение в различных областях:

Медицина — обезличивание данных пациентов, медицинских записей и результатов анализов необходимо для соблюдения норм конфиденциальности и защиты персональных данных. Это также позволяет использовать данные для научных исследований и статистики.

Финансовый сектор — банки и другие финансовые учреждения используют обезличивание для защиты данных клиентов, таких как номера счетов, транзакции и другие финансовые данные. Это важно, как для соблюдения законодательства (например, в рамках GDPR), так и для предотвращения утечек данных.

Маркетинг и реклама — компании, занимающиеся анализом поведения пользователей, используют обезличенные данные для изучения потребительских предпочтений и разработки рекламных стратегий, при этом защищая конфиденциальность клиентов.

Государственные структуры — обезличивание данных используется для обеспечения конфиденциальности при проведении статистических исследований и обработке персональных данных граждан [7].

1.6 Проблемы в области обезличивания данных

Несмотря на развитие методов обезличивания, существует несколько проблем, связанных с их применением:

Реидентификация данных — в некоторых случаях возможно восстановление исходной информации, особенно когда данные подвергаются многократному анализу

или объединению с другими источниками. Это особенно актуально в случае использования большого количества открытых данных, что требует дополнительных усилий для обеспечения безопасности.

Баланс между обезличиванием и полезностью данных — слишком сильное обезличивание может привести к потере полезности данных для анализа. Задача заключается в том, чтобы минимизировать потерю информации при защите конфиденциальности.

Законодательные требования — требования по обезличиванию данных, предъявляемые различными странами и регионами, могут существенно различаться. Это создает дополнительные сложности для международных компаний и организаций [8].

Еще одна проблема персональных данных заключается в том, что тяжело определить какая взаимосвязь между различными атрибутами позволяет однозначно идентифицировать субъекта, а какая нет. Для пациентов мед. учреждений совокупность атрибутов изображена на рисунке 2.6.1.

Список возможных совокупностей PII-атрибутов, обеспечивающих однозначную идентификацию пациента	Список возможных совокупностей PII-атрибутов, на основе которых невозможно однозначно идентифицировать пациента
1. Ф.И.О. + дата и место рождения	1. Ф.И.О. + информация об образовании
2. Ф.И.О. + адрес места регистрации	2. Ф.И.О. + место работы
3. Ф.И.О. + номер медицинской карты (ЭМК)	3. Ф.И.О. + адрес электронной почты
4. дата и место рождения + адрес места регистрации	4. адрес места регистрации + гражданство
5. Ф.И.О. + дата и место рождения + адрес проживания (реальный)	5. адрес места регистрации + информация об образовании
6. Ф.И.О. + дата и место рождения + гражданство	6. адрес места регистрации + адрес электронной почты
7. Ф.И.О. + дата и место рождения + информация об образовании	7. адрес проживания (реальный) + гражданство
8. Ф.И.О. + дата и место рождения + место работы + должность	8. адрес проживания (реальный) + информация об образовании
9. Ф.И.О. + дата и место рождения + сведения о законных представителях, членах семьи, родственниках и т. д.	9. адрес проживания (реальный) + место работы
10. Ф.И.О. + дата и место рождения + номер медицинской карты (ЭМК)	10. адрес проживания (реальный) + сведения о законных представителях, членах семьи, родственниках и т. д.

Рисунок 2.6.1 Совокупность атрибутов персональных данных для пациентов мед. учреждений

1.7 Выводы по главе

Обезличивание данных является важным инструментом в области защиты персональной информации. Современные методы обезличивания помогают минимизировать риски утечек данных, обеспечивая при этом возможность их использования для анализа и исследований. Однако, несмотря на значительный прогресс в этой области, проблема сохранения конфиденциальности и предотвращения реидентификации данных остается актуальной и требует дальнейших исследований и совершенствования методов обезличивания. В данной главе рассмотрены основные теоретические аспекты сферы обезличивания данных, включая цели, задачи, методы обезличивания данных и проблемы в данной области.

2. Анализ существующих систем

В последние годы разработка и внедрение систем и алгоритмов обезличивания данных стало важной частью обеспечения безопасности и конфиденциальности при обработке больших объемов информации. Современные подходы к обезличиванию охватывают различные методы и технологии, которые позволяют минимизировать риски утечек персональных данных, при этом сохраняя аналитическую ценность данных. В этой главе будет рассмотрен обзор существующих систем и алгоритмов обезличивания, а также их преимущества и ограничения.

Обезличивание данных может быть реализовано с использованием различных программных и аппаратных решений. Существующие системы можно разделить на несколько типов в зависимости от их архитектуры, области применения и используемых методов.

Рассмотрим существующие системы с позиций реализованных методов обезличивания, поддержки СУБД, открытости программного кода, наличия средств автоматизации, интеграция в бизнес процессы [9].

2.1 Системы для обезличивания данных

В PostgreSQL есть готовое решение по обезличиванию данных — библиотека PostgreSQL Anonymizer. В проекте реализован декларативный подход к обезличиванию. Это означает, что вы можете объявить правила обезличивания с помощью языка определения данных PostgreSQL (DDL) и указать свою стратегию обезличивания внутри самого определения таблицы.

В PostgreSQL существует готовое решение для обеспечения обезличивания данных — библиотека PostgreSQL Anonymizer.

Эта библиотека предоставляет возможность определения правил обезличивания с использованием языка определения данных PostgreSQL (DDL). Таким образом, возможно задать стратегию защиты непосредственно в определении таблицы.

Одним из минусов PostgreSQL Anonymizer можно отметить то, что данная библиотека разрабатывалась не поддерживает российские символы, а также обладает

ограниченным количеством доступных методов обезличивания.

Существует схожее решение от Oracle в СУБД MySQL. Но несмотря на большой функционал и универсальность методов обезличивания в MySQL, код не является открытым и требует ежемесячной подписки.

Одним из популярных решений в этой области является система псевдонимизации данных "Anonify". Эта система позволяет преобразовывать идентифицирующие данные в уникальные псевдонимы с возможностью восстановления исходных данных с использованием ключа. Система легко встраивается в любые бизнес процессы, однако обладает очень скудным функционалом и платной подпиской.

Одним из решений является "Google Differential Privacy", платформа, предназначенная для интеграции методов дифференцированной приватности в аналитические и исследовательские приложения. Метод дифференцированной приватности активно используется для защиты данных в аналитических приложениях, например, в исследовательских проектах или в области социальных сетей.

Программные решения, основанные на этом подходе, добавляют случайный шум к данным, что делает невозможным извлечение точной информации о конкретных индивидах, но сохраняет общие статистические закономерности. Хотя система тоже является легко встраиваемой, использование одного метода обезличивания, без его возможности изменения, является значительным минусом в его сравнении с другими продуктами.

На рынке существуют и другое платное программное обеспечение: K2View Data Masking, DATPROF Data Masking Tool, IRI FieldShield (Profile/Mask/Test), Accutiv Data Discovery & Masking, IRI DarkShield (Unstructured Data Masking), IRI CellShield EE (Excel Data Masking), Oracle - Data Masking and Subsetting, IBM InfoSphere Optim Data Privacy, Delphix и т.д.

Пожалуй, самым лучшим среди аналогов является система анонимизации "ARX Data Anonymization Tool". Это открытое программное обеспечение, которое предоставляет пользователям большое количество методов обезличивания. ARX поддерживает множество форматов данных и подходит для анонимизации больших

объемов информации, при этом обеспечивая возможность настраиваемых уровней конфиденциальности в зависимости от требований. Однако, система имеет высокий порог входа для новых пользователей и в основном предназначена для исследовательских, либо неавтоматизированных работ, т.к. не обладает API для интеграции с существующими бизнес процессами, а также в ней не реализованы часть методов анонимизации [10].

Таблица 2.1 Общий сравнительный анализ существующих систем

Продукт	Количество реализованных методы	Поддержка вводных данных	Открытый исходный код	Продукт
PostgreSQL Anonymizer	5	SQL	Да	PostgreSQL Anonymizer
MySQL Anonymizer	7	SQL	Нет	MySQL Anonymizer
Anonify	2	SQL	Нет	Anonify
Google Differential Privacy	Только синтетические	SQL	Нет	Google Differential Privacy
ARX Data Anonymization Tool	3	JDBC, CSV, EXCEL	Да	ARX Data Anonymization Tool

Таблица 2.2 Сравнительный анализ существующих систем по методам анонимизации

Продукт/ Методы	PostgreSQL Anonymizer	MySQL Anonymizer	Anonify	Google Differential Privacy	ARX Data Anonymization
Введение идентификаторов	Есть	Есть	Есть	Нет	Нет

Декомпозиции	Есть	Есть	Нет	Нет	Нет
Перемешивание	Есть	Есть	Есть	Нет	Нет
Изменение состава или семантики	Нет	Нет	Нет	Нет	Нет
Обобщений	Нет	Есть	Нет	Нет	Есть
Кодирование сверху/снизу	Нет	Нет	Нет	Нет	Нет
Локальное подавление	Нет	Нет	Нет	Нет	Есть
Микро-агрегирование	Нет	Нет	Нет	Нет	Есть
Добавление шума	Нет	Есть	Нет	Нет	Нет
Округления	Нет	Нет	Нет	Нет	Нет

Крупнейшей проблемой всех зарубежных продуктов является их ограничение на распространение и работу на территории Российской Федерации в связи с санкциями. Поэтому требуется создать свой аналог системы обезличивания данных, не зависящий от иностранных кампаний

2.4 Выводы по главе

Анализ существующих систем обезличивания данных показывает, что на данный момент существует несколько решений, каждый из которых применим в зависимости от контекста и требований к безопасности. Системы анонимизации и псевдонимизации предоставляют удобные инструменты для защиты конфиденциальности данных, однако для повышения их эффективности необходимо учитывать современные вызовы в области безопасности. Также данные системы не обеспечивают достаточную гибкость и имеют значительные недостатки, например, платная подписка, ограниченный функционал, набор методов, который не полностью соответствует требованиям Российского законодательства. Это является достаточным поводом для написания собственного программного обеспечения.

3. Практическая работа и технологические задачи

3.1 Анализ требований

Разработанное программное обеспечение предназначено для системы «Интеллектуальное управление, обогащение и обезличивание данных», разрабатываемой в рамках платформы «Доверенная среда обмена информации» (проекты Центра компетенций НГТУ). Основная задача программного обеспечения – облегчить настройку и обеспечить обезличивание реляционной базы данных. Помимо этого, программа будет предоставлять оценку информационных потерь и риска раскрытия информации.

Разрабатываемая программная система должна обладать следующим функционалом:

- подготовка данных для обезличивания (выбор типа атрибута и метода заполнения пропусков);
- выбор и настройка методов обезличивания;
- выбор методов оценки рисков;
- выбор методов оценки информационных потерь;
- загрузка и сохранения конфигурации для обезличивания базы данных;
- перенастройка уже сконфигурированных методов обезличивания;
- проведения процесса обезличивания данных;
- отображение времени, за которое было выполнено обезличивание данных;
- вывод метрик оценки информационных потерь и риска раскрытия информации.

Так же программа должна удовлетворять следующим нефункциональным требованиям:

- русская локализация интерфейса;
- наличие графического интерфейса;
- наличие в программе только одного главного окна;
- интуитивно понятный интерфейс.

3.2 Проектирование программного обеспечения

3.2.1 Описание классов

Разработанная программная система представляет собой сложную структуру, основанную на классах графического интерфейса, каждый из которых занимается отображением различных окон приложения, обеспечивая пользователю доступ к необходимой информации и возможность взаимодействия. Эти классы не только отображают информацию, но и управляют пользовательским вводом и реагируют на его действия.

Важной частью этой системы являются классы, которые используют классы графического интерфейса. Например, среди этих классов присутствует класс, ответственный за взаимодействие с базой данных. Он осуществляет получение информации о структуре таблиц и их содержимом.

Еще одним важным аспектом является класс, содержащий методы для подготовки и обезличивания данных. Этот класс заботится о конфигурации методов обезличивания, оценке потенциальных информационных потерь и рисков раскрытия информации.

Для обеспечения процесса обезличивания данных в базе данных был разработан специализированный интерфейс. Классы, реализующие этот интерфейс, предназначены для конкретизации методов обезличивания, обеспечивая гибкость этого процесса в соответствии с требованиями и потребностями конкретного пользователя.

Дополнительный класс, созданный для взаимодействия с базой данных, играет ключевую роль в обеспечении коммуникации между программой и хранилищем данных. Он содержит реализацию всех необходимых методов, обеспечивающих доступ к данным, и обеспечивает надежную и эффективную работу программы с базой данных.

Подробную UML-диаграмму можно увидеть на рисунке 3.1.

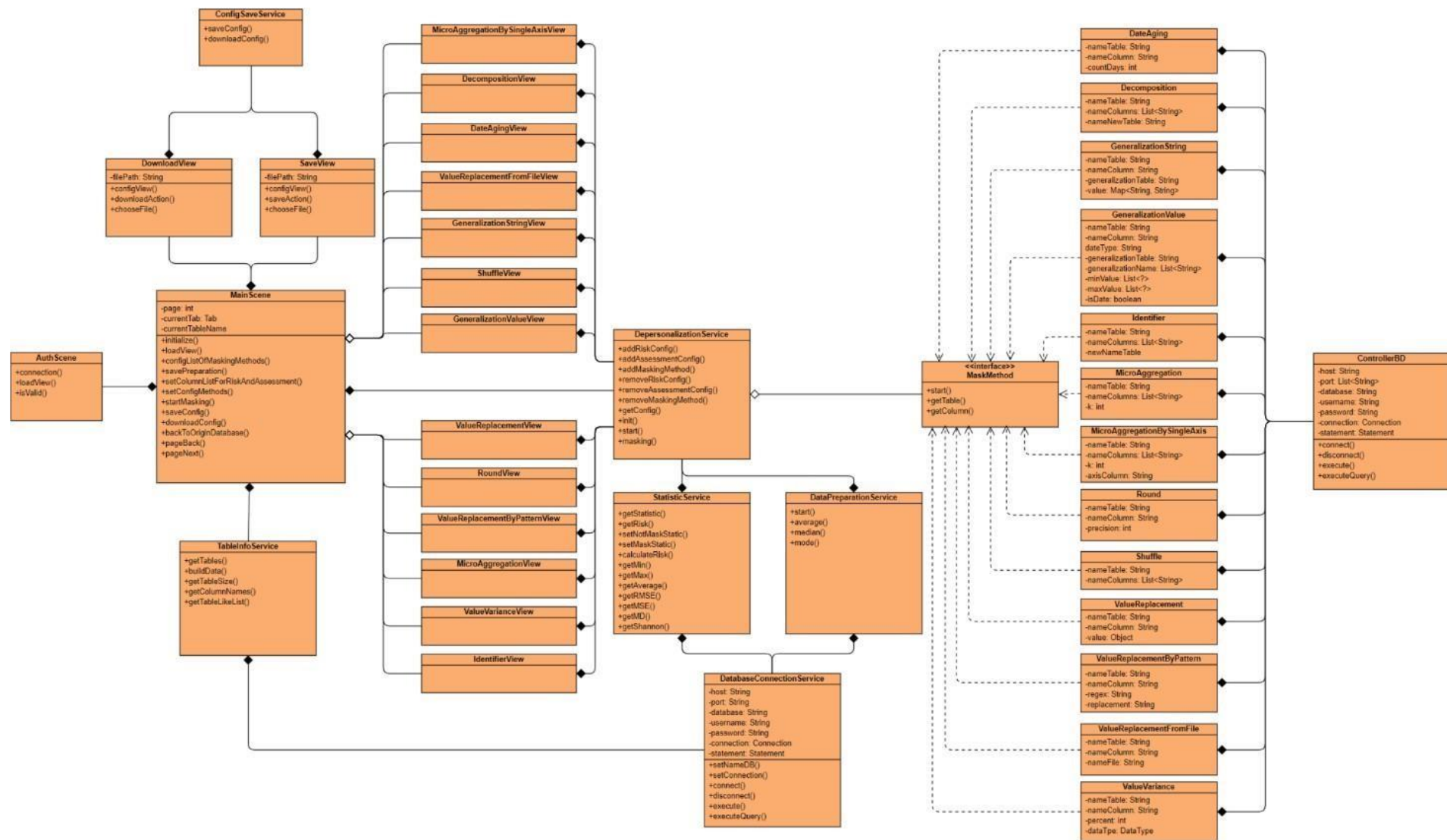


Рисунок 3.1 – UML-диаграмма структуры классов

Таким образом, разработанная программа представляет собой комплексную систему, включающую в себя различные классы и интерфейсы, обеспечивающие удобство использования, защиту данных и эффективное взаимодействие с базой данных. Подробную информацию о классах можно увидеть в таблице 3.1.

Таблица 3.1 – Описание разработанных классов

Имя класс	Описание
ConfigSaveService	Класс необходимый для загрузки и сохранения конфигурационных файлов.
DatabaseConnectionService	Класс необходимый взаимодействия с базой данных.
DataPreparationService	Класс реализующий модуль подготовки данных.
DepersonalizationService	Класс реализующий модуль обезличивания данных.
StatisticService	Класс собирающий статистику, данный класс реализует модуль оценки рисков раскрытия информации и оценку потерь информации.
TableInfoService	Класс преобразующий информацию в удобный формат.
AuthScene	Класс отображающий окно, содержащие поля, необходимые для установки соединения с базой данных.
DownloadView	Класс отображающий окно, загрузки файла с конфигурацией.
SaveView	Класс отображающий окно, сохранения файла с конфигурацией.
MainScene	Класс отображающий главное окно, в данном окне происходит просмотр таблиц и настройка методов обезличивания, оценки потерь, оценки рисков и начальная подготовка данных.
DateAgingView	Класс отображающий окно, необходимое для конфигурации метода старения данных.
DecompositionView	Класс отображающий окно,

	необходимое для конфигурации метода декомпозиции.
GeneralizationStringView	Класс, отображающий окно, необходимое для конфигурации метода обобщения для количественных признаков или дат.
GeneralizationValueView	Класс, отображающий окно, необходимое для конфигурации метода обобщения для качественных признаков.
IdentifierView	Класс, отображающий окно, необходимое для конфигурации метода введения идентификатора.
MicroAggregationView	Класс, отображающий окно, необходимое для конфигурации метода микроагрегирования по выбранным атрибутам.
MicroAggregationBySingleAxisView	Класс, отображающий окно, необходимое для конфигурации метода микроагрегирования по выбранным атрибутам путем сортировки одного из них.
RoundView	Класс, отображающий окно, необходимое для конфигурации метода округления.
ShuffleView	Класс, отображающий окно, необходимое для конфигурации метода перемешивания.
ValueReplacementView	Класс, отображающий окно, необходимое для конфигурации метода замены значений атрибута.
ValueReplacementByPatternView	Класс, отображающий окно, необходимое для конфигурации метода замены значений атрибута согласно заданному образцу (паттерну).
ValueReplacementFromFileView	Класс, отображающий окно, необходимое для конфигурации метода замены значений атрибута на случайное значение из файла.
ValueVarianceView	Класс, отображающий окно, необходимое для конфигурации метода добавления шума.
DateAging	Класс содержащий реализацию

	метода старения данных.
Decomposition	Класс содержащий реализацию метода декомпозиции.
GeneralizationString	Класс содержащий реализацию метода обобщения для количественных признаков или дат.
GeneralizationValue	Класс содержащий реализацию метода обобщения для качественных признаков.
Identifier	Класс содержащий реализацию метода введения идентификатора.
MicroAggregation	Класс содержащий реализацию метода микроагрегирования по выбранным атрибутам.
MicroAggregationBySingleAxis	Класс содержащий реализацию метода микроагрегирования по выбранным атрибутам путем сортировки одного из них.
Round	Класс содержащий реализацию метода округления.
Shuffle	Класс содержащий реализацию метода перемешивания.
ValueReplacement	Класс содержащий реализацию метода замены значений атрибута.
ValueReplacementByPattern	Класс содержащий реализацию метода замены значений атрибута согласно заданному образцу (паттерну).
ValueReplacementFromFile	Класс содержащий реализацию метода замены значений атрибута на случайное значение из файла.
ValueVariance	Класс содержащий реализацию метода добавления шума.
ControllerDB	Внутренний класс, который используется в методах обезличивания, для взаимодействия с базой данных.

3.2.2 Архитектура программного обеспечения

На рисунке 3.2 представлена архитектура программного решения, состоящая из пяти основных компонентов. Первый компонент - пользовательское приложение, разработанное на основе Spring Boot и JavaFX, и

отвечает за создание интерфейса пользователя и организацию взаимодействия с данными. Следующие четыре модуля выполняют различные функциональные задачи в рамках программы. Каждый модуль специализируется на определенной области функциональности и взаимодействует с остальными компонентами для обеспечения целостного функционирования программы. Такая архитектура позволяет эффективно структурировать программное решение и обеспечить его гибкость и масштабируемость.



Рисунок 3.2 – Архитектура ПО для решения задачи обезличивания данных

На рисунке 3.3 показана структура программы по обезличиванию данных. Структура состоит из графического интерфейса, модуля загрузки данных, модуля подготовки данных перед проведением процедуры обезличивания, модуля обезличивания, модуля оценки риска раскрытия информации и модуля оценки информационных потерь.



Рисунок 3.3 – Структурная схема программного обеспечения

Графический интерфейс представляет собой важный компонент системы, обеспечивающий наглядное отображение данных в виде таблиц и предоставляющий пользователю доступ к результатам обезличивания.

Модуль "Подготовка данных перед проведением процедуры обезличивания" осуществляет заполнение пропусков в атрибутах, определенных пользователем, что обеспечивает более точные и надежные результаты обезличивания.

Модуль обезличивания играет ключевую роль в процессе создания новой базы данных, которая сохраняет структуру оригинальной базы, но при этом внедряет выбранные методы обезличивания, обеспечивая сохранение конфиденциальности данных. Одновременно ведется мониторинг времени, затраченного на процесс обезличивания, что позволяет оценить эффективность методов.

Модуль оценки риска раскрытия информации проводит анализ вероятности деобезличивания данных по атрибутам, определенным пользователями, что способствует выявлению потенциальных уязвимостей системы.

Модуль оценки информационных потерь осуществляет сопоставление исходной и обезличенной баз данных с целью определения объема утраченной информации, что помогает оценить степень сохранности данных после процедуры обезличивания.

3.2.3 Алгоритм решения задачи обезличивания данных

Был разработан программный алгоритм обезличивания данных, который реализован в программной системе (рисунок 3.4).

Основная цель данного алгоритма заключается в обеспечении высокой эффективности процесса обезличивания. Для достижения этой цели алгоритм включает несколько этапов решения задачи, что обеспечивает комплексный подход к решению проблемы.

Этапы алгоритма обезличивания включают в себя ряд ключевых шагов, которые выполняются последовательно.

1. Создание копии оригинальной базы данных: Начальным этапом является создание дубликата исходной базы данных, что обеспечивает сохранность исходных данных в случае необходимости.

2. Анализ статистических данных атрибутов: Проводится расчет и анализ статистических характеристик атрибутов в оригинальной базе данных, которые планируется обезличить или которые требуются для дальнейшего исследования.

3. Предварительная обработка данных: На этом этапе происходит заполнение пропусков в атрибутах и определение типов атрибутов.

4. Применение методов обезличивания: На данном этапе применяются различные методы обезличивания данных согласно установленным целям и требованиям проекта. Это может включать в себя замену значений атрибутов, удаление идентифицирующих признаков или применение декомпозиции.

5. Расчет времени выполнения методов обезличивания: После завершения обезличивания проводится оценка времени выполнения, что позволяет оценить их эффективность и производительность.

6. Оценка рисков раскрытия информации: Применяются методы для оценки вероятности раскрытия информации после обезличивания.

7. Анализ статистических данных обезличенной БД: Проводится анализ статистических данных атрибутов в обезличенной базе данных.

8. Оценка информационных потерь: На основе сравнения статистических данных оригинальной и обезличенной баз данных производится оценка потерь информации.

9. Предоставление результатов пользователю: Наконец, пользователю предоставляются результаты обезличивания для дальнейшего использования.

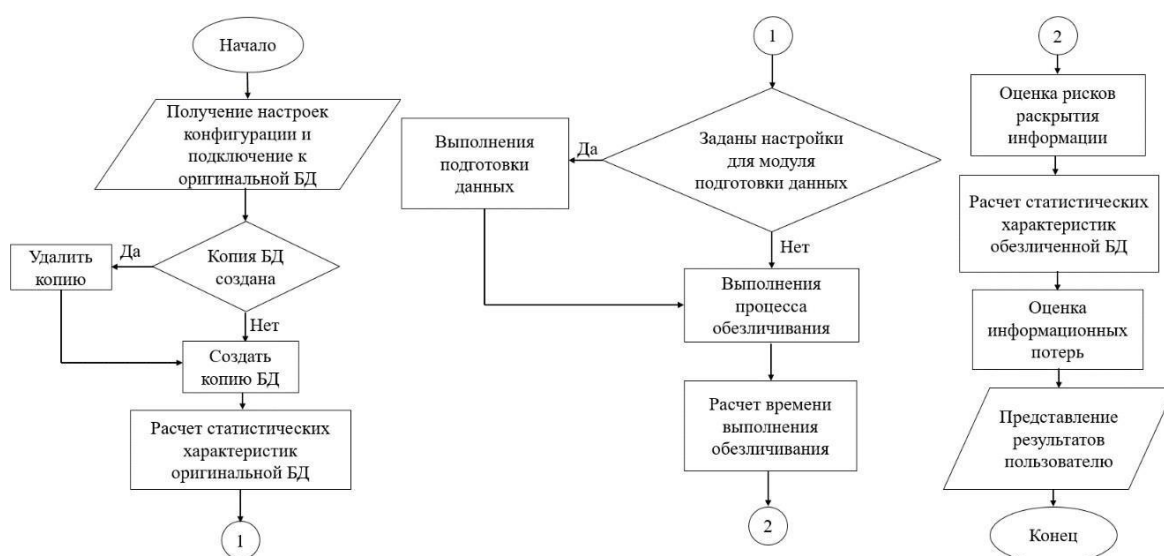


Рисунок 3.4 – Блок-схема алгоритма решения задачи обезличивания данных

3.3 Интерфейс программной системы

Графическая часть программы предполагает взаимодействие с пользователем через графический интерфейс в виде оконной формы настольного приложения. Приложение реализовано в виде одного главного окна, и вспомогательных окон, служащих для сохранения и загрузки конфигурации и настройки методов обезличивания.

Из главного меню возможны действия:

- просмотр базы данных;
- переход по 4 вкладкам, которые представляют модули программы (подготовка данных, обезличивания, риск, оценка потерь).
- переход на вкладку с конфигурацией;
- открытие окон, каждое из которых отвечает за конфигурирование своего метода;
- открытие окон для сохранения и загрузки конфигурации.

На рисунке 3.5 представлены все возможные варианты использования программы.

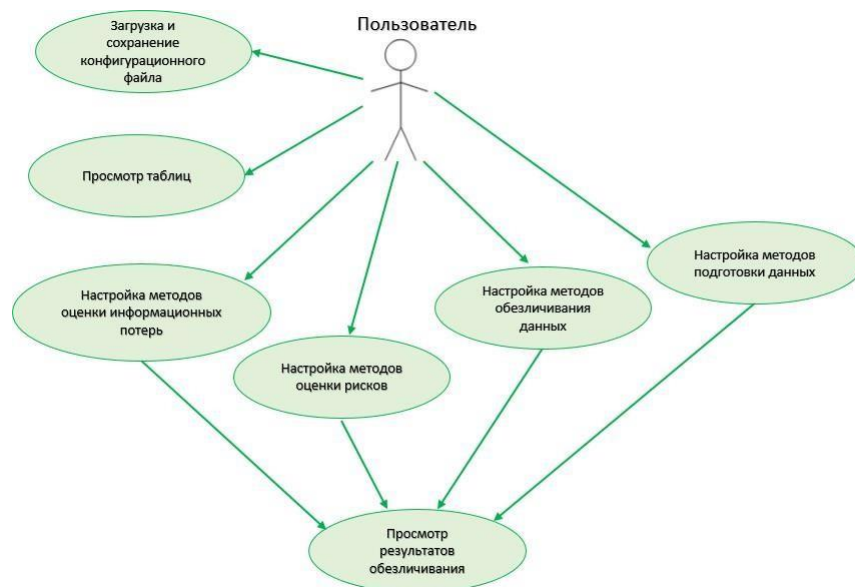


Рисунок 3.5 – Общая схема работы приложения

С учетом функциональных и нефункциональных требований был создан следующий прототип-макет графического интерфейса.

Разработанная программа имеет возможность подключиться к базам данных PostgreSQL (рисунок 3.6).

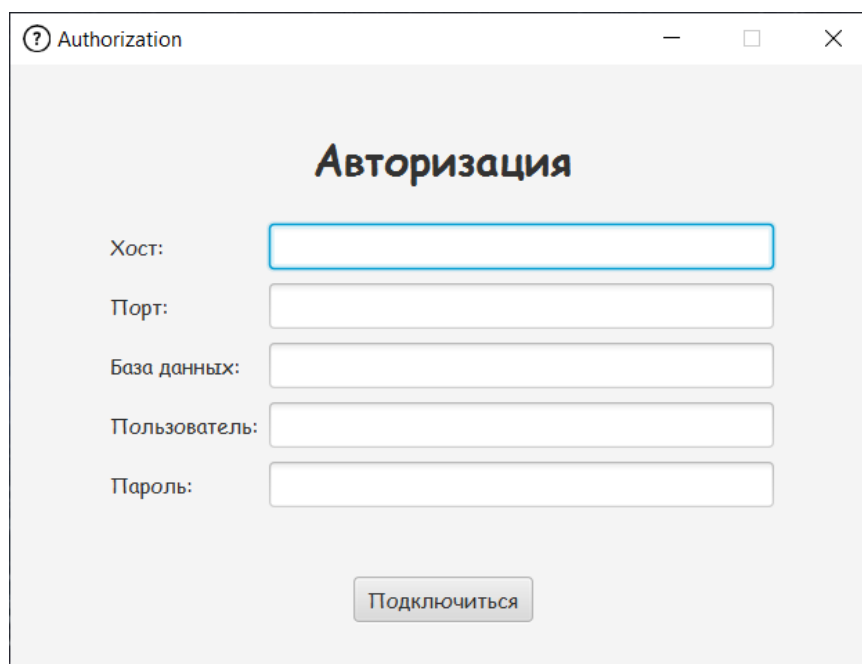


Рисунок 3.6 – Окно для подключения к базе данных
Основное окно программы (рисунок 3.7). Оно содержит следующие блоки:

- панель для отображения данных;
- верхняя панель;
- боковая правая панель содержит 5 вкладок, каждая вкладка, отвечает за один модуль программы, кроме вкладки конфигурации.

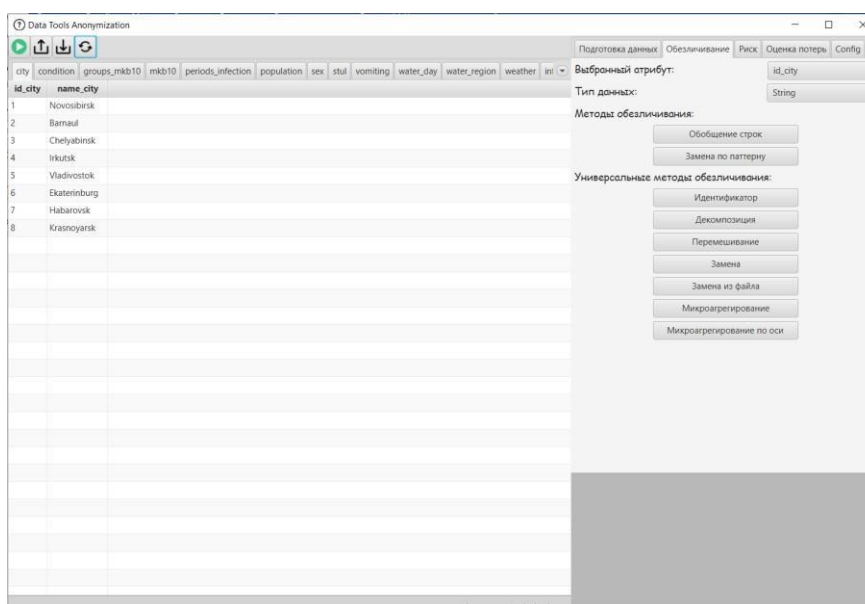


Рисунок 3.7 – Окно графического интерфейса программы

Рассмотрим подробнее панели:

[illegible]

В верхней части пользовательского интерфейса размещены четыре кнопки, как показано на рисунке 3.9. Первая кнопка инициирует процесс обезличивания данных. После запуска процесса появляется окно, сообщающее о текущем статусе обезличивания, как показано на рисунке 3.10. По завершении обезличивания в этом окне отображается время, затраченное на процесс, как показано на рисунке 3.11.

Последняя кнопка предназначена для обновления программы. При её нажатии происходит возврат к отображению оригинальной базы данных. Это

важно после обезличивания, так как программа начинает отображать обезличенную базу. Кроме того, эта кнопка служит защитой от случайного повторного обезличивания уже обезличенной базы данных, предотвращая потенциальные ошибки и потерю данных.



Рисунок 3.9 – Верхняя панель

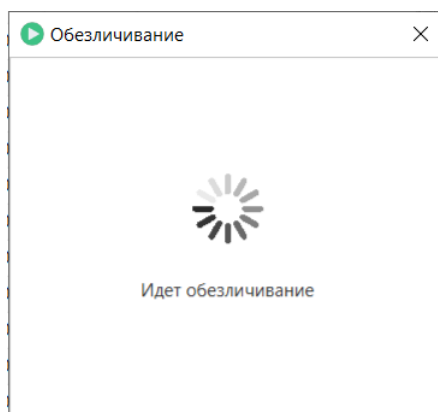


Рисунок 3.10 – Окно, сигнализирующее о процессе обезличивания

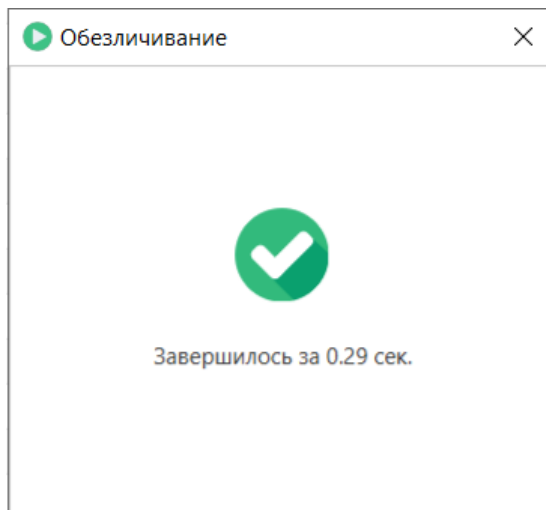


Рисунок 3.11 – Окно, после окончания процесса обезличивания

Последняя панель программы является боковая (рисунок 3.12). Данная панель делится на нижнюю и верхнюю.

Подготовка данных Обезличивание Риск Оценка потерь Config

Выбранный атрибут: id_city

Тип: Insensitive

Метод заполнения поля: none

Статистика по обезличиванию

MSE: 0.9989960689811811
MD: 8.969871474578262
Shannon: 45.734561707076374
ProsecutorMetricB: 0.2
ProsecutorMetricC: 0.19444444444444445

Рисунок 3.12 – Боковая панель вкладки «Подготовка данных»

В нижней части боковой панели представлены статистические данные, относящиеся к обезличенной базе данных. Эти данные включают в себя метрики оценки информационных потерь, которые представляют собой отношения между значениями оригинальной базы данных и обезличенной версии. Пользователь имеет возможность выбирать нужные метрики оценки потерь информации, такие как минимальное, максимальное, среднее значение, среднеквадратичное отклонение (RMSE), среднеквадратическая ошибка (MSE), медианное значение (MD) и индекс Шеннона.

Кроме того, также предоставляется выбор методов оценки рисков, таких как ProsecutorMetricA, ProsecutorMetricB, ProsecutorMetricC и GlobalRisk. Эти методы позволяют пользователю анализировать возможные риски, связанные с обезличиванием данных, и принимать информированные решения о дальнейших шагах в обработке данных.

В верхней части боковой панели расположены 5 вкладок, каждая из которых представляет собой отдельный модуль функциональности: подготовка данных, обезличивание, оценка рисков и потерь, а также конфигурация.

Первая вкладка, представленная в виде модуля подготовки данных, открывает доступ к возможности назначения выбранному атрибуту соответствующего типа данных (например, insensitive, sensitive, quasi-sensitive, identifying). Кроме того, пользователь может выбрать метод заполнения пропущенных значений для указанного атрибута из предложенных вариантов (например, none, average, median, mode). Эти опции обеспечивают пользователю контроль над предварительной обработкой данных перед обезличиванием.

Во второй вкладке располагается модуль обезличивания данных, представленный на рисунке 3.13. Здесь пользователь имеет возможность назначить тип атрибута данных, а также применить один или несколько методов обезличивания.

Подготовка данных | Обезличивание | Риск | Оценка потерь | Config

Выбранный атрибут: id_city

Тип данных: String

Методы обезличивания:

- Обобщение строк
- Замена по паттерну

Универсальные методы обезличивания:

- Идентификатор
- Декомпозиция
- Перемешивание
- Замена
- Замена из файла
- Микроагрегирование
- Микроагрегирование по оси

Рисунок 3.13 – Боковая панель вкладки «Обезличивание»

Существует два вида методов обезличивания: универсальные, которые могут быть применены к атрибутам независимо от их типа данных, и специализированные, предназначенные только для определенных типов данных (String, Integer, Float или Date).

К универсальным методам обезличивания относятся:

- Идентификатор
- Декомпозиция

- Перемешивание
- Замена
- Микроагрегирование

Методы обезличивания для строковых атрибутов включают:

- Обобщение
- Замена по шаблону

Для целочисленных, вещественных и датовых атрибутов доступны методы:

- Обобщение
- Шум

Для вещественных атрибутов также можно применить метод округления, а для даты - округление и старение даты. Эти методы обеспечивают разнообразные способы обезличивания данных в зависимости от их типа и требований безопасности.

3.4 Выводы по главе

В данной главе было описано программное обеспечение, предназначенное для поддержки процесса обезличивания реляционных баз данных и предоставления оценки информационных потерь и риска раскрытия информации. Оно должно обладать функциями подготовки данных, выбора методов обезличивания и оценки рисков, а также удовлетворять нефункциональным требованиям, таким как русскоязычный интерфейс и интуитивная навигация.

При разработке архитектуры программы использовался паттерн MVC (модель, вид, контроль). Модель представляет собой классы, отвечающие за методы обезличивания данных. Представление включает классы графического интерфейса, отображающие информацию пользователю и обеспечивающие взаимодействие. Контроллер выступает посредником между моделью и пользовательским интерфейсом, управляя вызовами методов модели и управлением потоком данных в приложении. Был представлен интерфейс программы и описаны основные окна разработанной программы.

ЗАКЛЮЧЕНИЕ

Обезличивание данных является важной и неотъемлемой частью современных практик защиты персональной информации. В условиях быстрого роста объемов данных и ужесточения требований к их безопасности, методы обезличивания помогают минимизировать риски утечек личной информации и обеспечивают возможность использования данных в аналитических, исследовательских и коммерческих целях без угрозы нарушения конфиденциальности.

В ходе работы были рассмотрены основные теоретические аспекты сферы обезличивания данных, включая цели, задачи, методы обезличивания данных и проблемы в данной области, а также существующие системы, обеспечивающие их реализацию. Каждый из этих методов имеет свои особенности и области применения, что позволяет гибко подходить к решению задач защиты данных в различных сферах, включая здравоохранение, финансы, маркетинг и государственное управление.

Однако, несмотря на развитие технологий обезличивания, остаются важные проблемы, связанные с возможностью реидентификации данных, потерей полезности информации и различиями в законодательных требованиях различных стран. Эти вопросы требуют постоянного совершенствования методов и алгоритмов обезличивания, а также разработки новых подходов, которые смогут эффективно балансировать между защитой данных и их аналитической ценностью.

Таким образом, обезличивание данных остается актуальной и необходимой задачей для обеспечения безопасности и конфиденциальности в эпоху цифровизации. Решение этих задач требует комплексного подхода, использования различных технологий и методов, а также соблюдения высоких стандартов безопасности и соответствия нормативным требованиям.

СПИСОК ИСТОЧНИКОВ

1. НГТУ - ВТ - О кафедре [Электронный ресурс] URL: <https://ciu.nstu.ru/kaf/vt> (дата обращения: 20.10.24)
2. Макарова Е.А., Андрейченко А.Е., Казакова М. А. Обезличивание медицинских текстовых данных для разработки и внедрения систем искусственного интеллекта // Правовая информатика, 2024. - С. 96-104. (Перевод статьи)
3. Soumia Zohra El Mestari., Gabriele Lenzini, Huseyin Demirci Preserving data privacy in machine learning systems // Computers & Security. - 2024. – V.137.
4. Guangxi Lu, Kaiyang Li, Xiaotong Wang Neural-based inexact graph de-anonymization// High-Confidence Computing – 2024. – V. 4, № 1.
5. Mark Elliot, Functional anonymisation: Personal data and the data environment // Computer Law & Security Review, 2018, P. 204-221.
6. Guangxi Lu, Kaiyang Li, Xiaotong Wang Anonymisation of personal data – A missed opportunity for the European Commission // High-Confidence Computing. – 2024. – V. 4, № 1.
7. А. В. Борисов, А. В. Босов А. В. Иванов Применение имитационного компьютерного моделирования к задаче обезличивания персональных данных. Модель и алгоритм обезличивания методом синтеза.// Программирование – 2023 - № 5. - С. 19-34.
8. Столбов А.П. О стандартизации методов псевдонимизации персональных данных в здравоохранении // Молодой ученый. - 2023. - С. 5-13.
9. David Nettleton Data Privacy and Privacy-Preserving Data Publishing. - Commercial Data Mining, 2014, P 217-228
10. Nihad Ahmad Hassan, Rami Hijazi Data Hiding Using Encryption Techniques. - Data Hiding Techniques in Windows OS., 2017, P. 133-205